University of Bayreuth

Reinforcement Learning

Assessed Coursework 2 – Deep Q-Networks

Environment
CartPole-v1

Name	[Sonika —]
BT Code	[bt729624]
Department	[Philosophy and Computer Science]
Course	Reinforcement Learning
Date	April 2, 2026

Question 1: Tuning the DQN

1.1 Hyperparameters

The following key changes were done so that agent can learn effectively: adding hidden layers to the network architecture, increasing the replay buffer capacity, increasing the batch size, introducing an epsilon decay schedule, adjusting the target network update frequency, switching the optimiser, and adding a discount factor to the loss function.

During code inspection, two components in `utils.py` were also adjusted to better match the standard DQN formulation:

- **Forward pass:** The `forward()` method applied ReLU to every layer, including the final output layer. Since Q-values can take both positive and negative values, constraining the output with ReLU would distort the learning signal. ReLU activation was therefore applied only to the hidden layers, leaving the output layer linear.
- **Bellman loss:** The `loss()` function did not include a discount factor in the Bellman target:

$$Q(s_t, a) \leftarrow Q(s_t, a) + \alpha\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a)\big] \tag{1}$$

  The Bellman target $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$ from Equation 1 is the same target used in the DQN loss. Applying this update to the neural network setting, with a separate target network $\hat{Q}$ for stability and a terminal state mask, gives:

$$y_t \;=\; r_t \;+\; \gamma \cdot \max_{a'} \hat{Q}(s_{t+1},\, a') \cdot (1 - \mathbf{1}_{\text{terminal}}) \tag{2}$$

  where r_t is the immediate reward, $\gamma = 0.99$ is the discount factor, $\hat{Q}$ is the target network, and $(1 - \mathbf{1}_{\text{terminal}})$ equals zero at terminal states and one otherwise, ensuring no future reward is added when the episode ends.

Table 1 lists all hyperparameters used in the final tuned implementation.

Table 1: Hyperparameters used in the tuned DQN implementation.

Hyperparameter	**Value**	**Justification**
Architecture	`[4, 64, 64, 2]`	The original network had no hidden layers and could only learn linear relationships. Two hidden layers of 64 neurons each give the network enough depth to learn the non-linear patterns needed to reliably balance the pole.
Optimiser	Adam	SGD with lr=1.0 caused large erratic weight updates that destabilised training. Adam automatically adjusts the step size per parameter, giving smoother and more consistent learning.
Learning rate	`4e-3`	Chosen after experimentation. A higher rate caused oscillation, a lower rate was too slow to converge within 300 episodes. `4e-3` gave a good balance between speed and stability.

Hyperparameter	Value	Justification
Replay buffer size	50,000	With a buffer of just 1, the original agent re-learned from the same single experience repeatedly. A buffer of 50,000 ensures training draws from a wide mix of past situations, breaking temporal correlation.
Batch size	128	Training on one transition at a time produces extremely noisy gradient estimates. Sampling 128 transitions per update smooths this out considerably.
Min buffer (warm-up)	1,000	Training does not begin until 1,000 transitions are stored. This ensures the agent starts learning from a varied set of experiences rather than an almost-empty buffer.
Training frequency	Every 4 steps	Updating every 4 steps rather than every step gives the buffer more time to diversify between updates, reducing overfitting to the most recent experience.
Gamma (γ)	0.99	CartPole gives +1 reward per step survived. A high γ of 0.99 means the agent strongly values long-horizon survival. This term was missing entirely from the original loss function.
`EPS_START`	1.0	Starting fully random ensures the replay buffer is populated with a wide variety of experiences before any training begins.
`EPS_END`	0.01	A floor of 1% ensures the agent never completely stops exploring, preventing it from locking into a suboptimal policy.
`EPS_DECAY`	75 episodes	A decay of 50 was tried first but was too aggressive,the agent committed to an early poor policy before the buffer was diverse enough. Increasing to 75 gave a smoother transition from exploration to exploitation and more consistent results.
TAU (soft update)	0.01	Rather than hard-copying the target network every episode (which caused sudden jumps in the Bellman targets), a soft update gradually blends 1% of the policy weights into the target each step, keeping the training targets stable.

1.2 Exploration vs Exploitation

An epsilon-greedy strategy with exponential decay was used to balance exploration and exploitation. At the start of training, $\varepsilon = 1.0$, so the agent acts completely at random and builds up

a diverse replay buffer. As training progresses, epsilon decays exponentially from $\varepsilon_{\text{start}} = 1.0$ down to a floor of $\varepsilon_{\text{end}} = 0.01$, controlled by a decay constant τ.

The decay constant $\tau = 75$ was chosen based on the buffer warm-up threshold. With `MIN_BUFFER` = 1,000 and typical early episode lengths of 15–20 steps, the buffer fills after roughly 50–70 episodes, so it makes sense for ε to remain relatively high during this period so the buffer is populated with diverse transitions before exploitation kicks in.

Two values of τ were compared during tuning, as shown in Figure 1. At the buffer warm-up point around episode 70, $\tau = 50$ has already fallen to $\varepsilon \approx 0.25$, committing the agent to exploitation before the buffer is diverse enough. In contrast, $\tau = 75$ maintains $\varepsilon \approx 0.40$ at this point. In practice, $\tau = 50$ produced fast early learning but sometimes locked the agent into a suboptimal policy before the buffer was sufficiently diverse. Increasing τ to 75 gave a smoother and more reliable learning curve.

A floor of $\varepsilon_{\text{end}} = 0.01$ was kept rather than decaying all the way to zero. This small residual exploration prevents the agent from getting permanently stuck in a repetitive habit and maintains robustness in rarely visited states.

The original code set `EPSILON = 1` as a fixed constant, meaning the agent always acted randomly and never used its learned Q-values equivalent to having no learning at all. Introducing the exponential decay schedule is the single most impactful change made to the training loop.

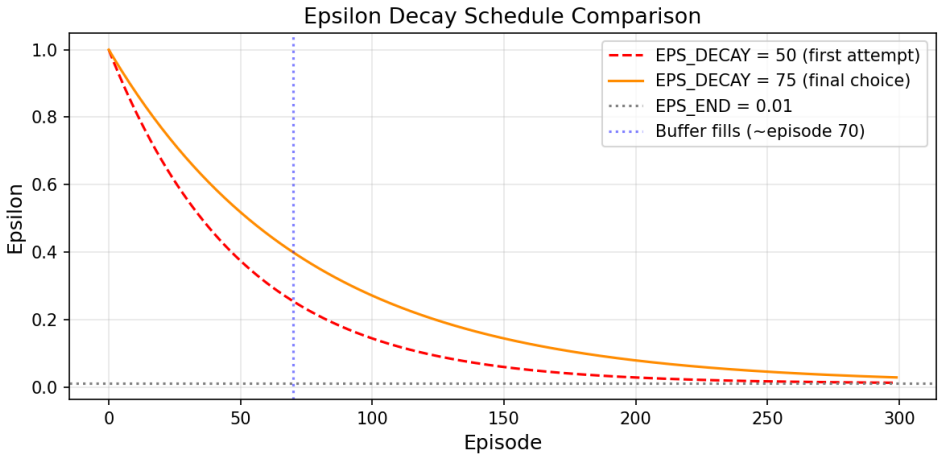

Figure 1: Comparison of epsilon decay schedules for $\tau = 50$ and $\tau = 75$. The vertical dotted line marks the point at which the replay buffer is approximately full (∼episode 70). At this point $\tau = 75$ maintains higher exploration, leading to a more diverse buffer before exploitation begins.

1.3 Learning Curve

The tuned DQN was trained 10 independent times from scratch, each with a freshly initialised policy network and an empty replay buffer. Figure 2 shows the mean return per episode averaged over all 10 runs, with the shaded region representing ±1 standard deviation.

The learning curve shows three distinct phases:

1. **Exploration phase** (episodes 0-130): The mean return stays flat near 20, reflecting the random baseline. Epsilon is still high and the buffer is filling up.

2. **Learning phase** (episodes 130-180): The return rises sharply as epsilon decays enough for the agent to start exploiting its learned Q-values.

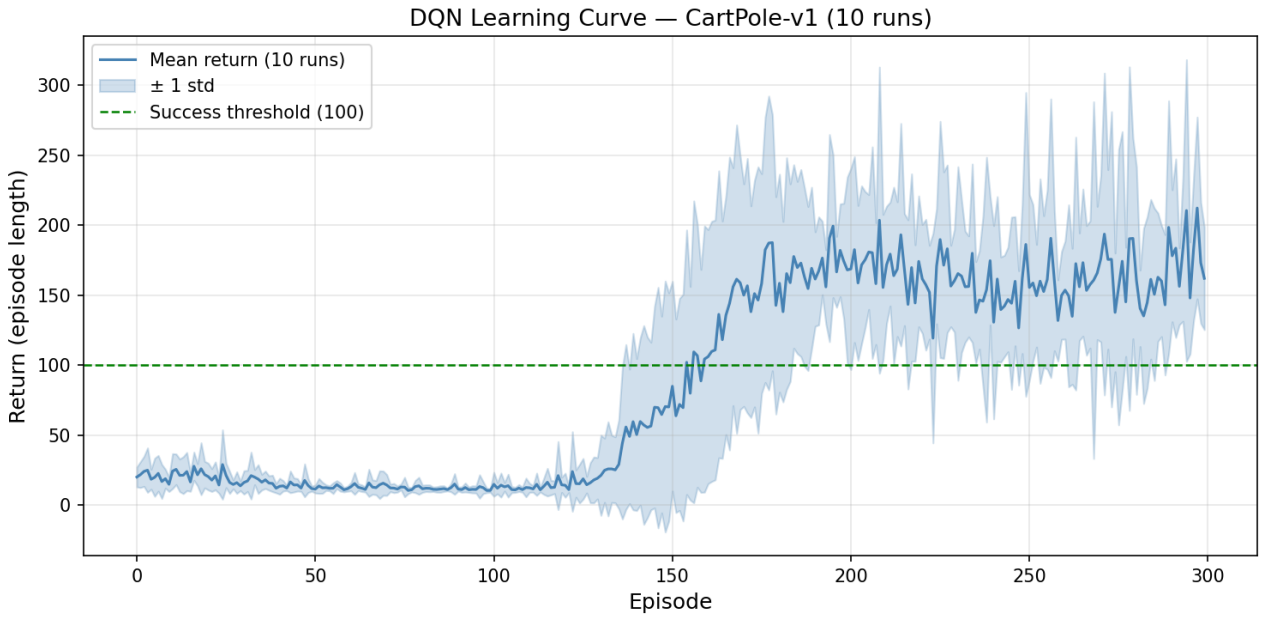

Figure 2: Mean return per episode over 10 independent training runs on CartPole-v1. Shaded region = ±1 standard deviation. The dashed green line marks the success threshold of 100.

3. **Stable phase** (episodes 180-300): The mean settles above 100 and stays there for the remainder of training, reaching 150-200 on average by episode 300.

The success criterion mean return above 100 for at least 50 consecutive episodes is clearly met. The mean crosses 100 at approximately episode 160 and does not fall below it again, giving over 100 consecutive episodes above threshold.

The standard deviation is narrow in the early phase since all runs behave similarly under high epsilon. It widens once learning begins, reflecting natural run-to-run variation in convergence speed. This is a well-known property of DQN training, driven by sensitivity to network initialisation and the random order of experience sampling.

Question 2: Visualising the DQN Policy

After training, the policy network from the final run was used to generate visualisations of the learned policy and Q-values. In all plots, the cart position is fixed at zero (centre of the track), the pole angle is on the x-axis, and the pole angular velocity is on the y-axis. Four cart velocities are examined: $v \in \{0,\ 0.5,\ 1.0,\ 2.0\}$.

2.1 Greedy Policy Slices

Figure 3 shows the greedy action chosen by the trained DQN across the pole angle and angular velocity state space for four fixed cart velocities.

At cart velocity $v = 0$, the decision boundary is roughly diagonal. When the pole leans right (positive angle) and falls further right (positive angular velocity), the agent pushes right to move the cart under the pole. When the pole leans or falls left, the agent pushes left. This is the physically correct corrective strategy, push in the direction the pole is tipping.

The diagonal boundary also shows the agent accounts for angular velocity, not just pole angle. If the pole is slightly to the right but has a strong leftward angular velocity (top-left region), the agent may already push left, anticipating the pole will swing back. This suggests the agent is not just reacting to the current pole angle but also accounting for which way it is already

moving.

As cart velocity increases from 0 to 2.0, the boundary shifts progressively. At $v = 2.0$, the push-right (yellow) region expands substantially the fast-moving cart must keep accelerating rightward to stay under the pole. This shift confirms that the policy depends on all four state dimensions simultaneously, not just the pole angle alone.

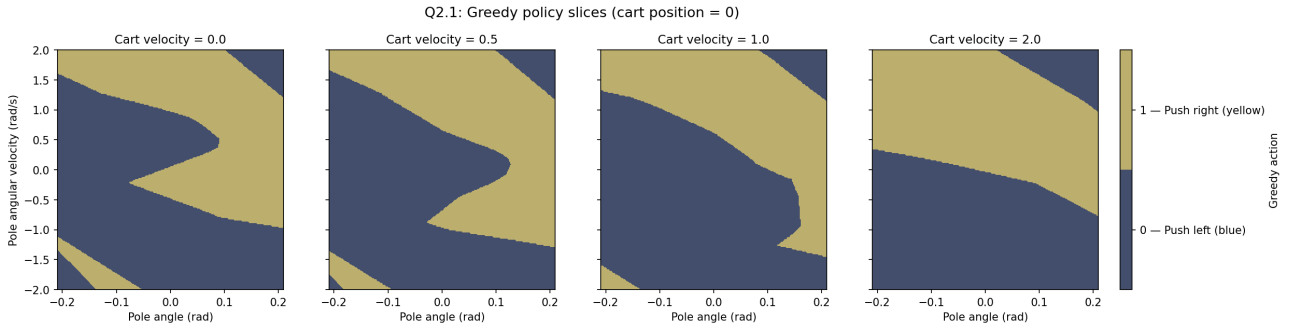

Figure 3: Greedy policy slices for four values of cart velocity with cart position fixed at zero. **Blue** = action 0 (push left) **yellow** = action 1 (push right).

2.2 Greedy Q-Value Slices

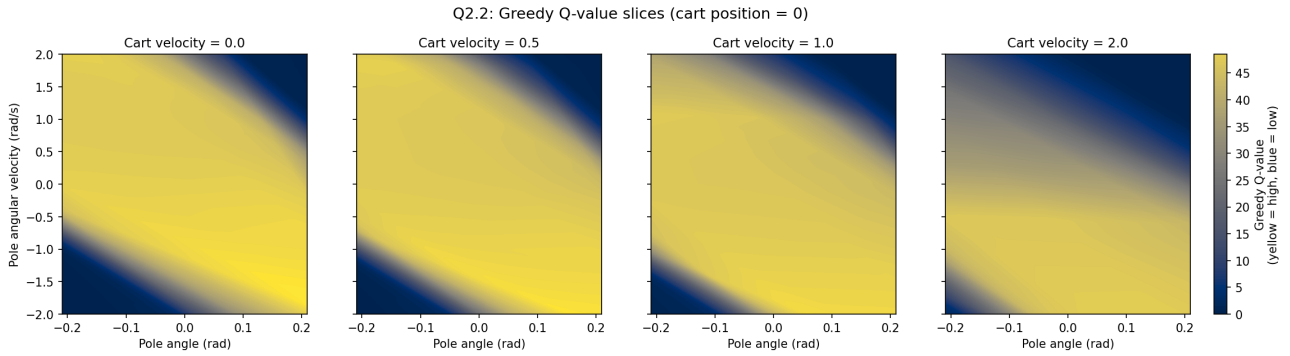

Figure 4: Greedy Q-value slices for four values of cart velocity with cart position fixed at zero. **Yellow** = high Q-value (safe states) **blue** = low Q-value (dangerous states near termination). Cividis colourmap. Shared colour scale across all four subplots.

Figure 4 shows $\max_a Q(s, a)$ the greedy Q-value at each state. Yellow regions indicate states where the agent expects high cumulative reward. Blue regions indicate dangerous states where the episode is likely to end soon.

At $v = 0$, the highest Q-values appear in the centre, where pole angle and angular velocity are both small. Since CartPole gives a reward of $+1$ per step, the Q-value at any state is approximately the expected discounted return from that state:

$$R_t = \sum_{k=0}^{T} \gamma^k \cdot r_{t+k+1} \tag{3}$$

With $\gamma = 0.99$ and $r = +1$ at every step, states near the centre of the plot allow the agent to survive for many steps, so the sum in Equation 3 accumulates over a longer horizon and gives a high Q-value. States near the termination boundaries (± 0.2095 rad) force early episode termination, cutting the sum short and producing the low Q-values (blue) visible at the edges of each subplot.

The observed peak Q-values are roughly 35-45. This is somewhat lower than a naive estimate based on the average episode length of 150–200 steps would suggest, but this is expected as the plots fix cart position at zero and show slices of the state space rather than the actual distribution of states the agent visits most frequently during a successful episode. It is also worth noting that Q-values do not reach exactly zero at the termination boundaries, since the agent still receives a +1 reward on the final step before the episode ends, consistent with Equation 3.

The Q-value gradient is diagonal rather than symmetric. States where angle and angular velocity have opposite signs such as the pole leaning right but already rotating back left have higher Q-values than states where both point in the same direction. A pole that is already self-correcting allows the agent to survive longer, adding more terms to the sum in Equation 3. The DQN has captured this asymmetry, which is also consistent with the policy boundary seen in Figure 3.

As cart velocity increases, the high Q-value region shifts and peak values decrease. At $v = 2.0$, the safe region is noticeably smaller and overall Q-values are lower, reflecting the added difficulty of controlling a fast-moving cart. Even in states the agent considers relatively safe, the expected survival time is shorter when the cart is already moving quickly toward its own boundary (± 2.4), which directly shortens the sum in Equation 3 and reduces the Q-value.

Together, Figures 3 and 4 are consistent the greedy action chosen in each region of Figure 3 corresponds to the action that achieves the high Q-values seen in Figure 4, confirming that the DQN has learned a coherent and physically reasonable policy overall.

Annex: Declaration of Generative AI Use

Instructions: This form must be completed, signed, and included as the final page of your `coursework2report.pdf`*. Failure to complete this declaration or disclose all information related to your use of generative AI may result in a penalty or automatic failing grade.*

Student Name: Sonika

Student ID (BT Code): bt729624

Date: 02/04/2026

Please select the statement that best describes your use of Generative AI (e.g., ChatGPT, GitHub Copilot, Claude) in this coursework:

☐ **No AI Assistance:** I certify that I did not use any Generative AI tools to complete this assignment. All code and written analysis are entirely my own work.

☒ **AI Assistance (minimal use):** I used Generative AI tools to assist with my work. I have detailed the specific tools and how they were used below. I certify that I reviewed and understand all AI-generated content included in my submission.

If you used AI, please provide details below:

Which tools did you use? (e.g., ChatGPT-4, GitHub Copilot, etc.)

Claude and Chatgpt 4

How did you use them? (Check all that apply)

☒ Explaining concepts or debugging errors.

☐ Generating code snippets (which I modified/integrated).

☒ Rephrasing or editing text for the report.

☐ Generating entire functions or classes.

☐ Other: _____

Specific Prompts/Queries used (Provide 1–2 examples if applicable):

What does it mean physically when the pole angular velocity and pole angle have opposite signs, why would that be a safer state. Why does the Q-value drop near the termination boundaries of plus or minus 0.2095 radians. I don't understand why a replay buffer of size 1 makes learning impossible, can you explain

Declaration: I confirm that the information provided above is accurate. I understand that submitting work that I do not understand or cannot explain is a violation of academic integrity.

Signature: 

Date: 02/04/2026